# AI/ML Layer for the Karnataka Crime Intelligence Platform

## A Research Report on the Four-Model Architecture

---

## 1. Introduction

The crime analytics platform's intelligence layer is built around four complementary machine learning models, each addressing a different analytical question. None of these models works in isolation — they share a common feature store, and their outputs are combined in a serving layer before reaching the dashboard. This report walks through each model's purpose, the algorithm behind it, what data it needs, what it produces, and how it should be evaluated and governed.

The four models map directly onto the platform's stated capabilities:

| Model | Platform capability it powers |
|---|---|
| XGBoost (risk predictor) | Predictive risk scoring, district-level risk dashboards |
| ST-DBSCAN (hotspot detector) | Spatiotemporal crime hotspots, "red-zone" map overlays |
| Isolation Forest (anomaly detector) | Anomaly detection, flagging unusual FIRs for investigator review |
| Graph algorithm (network mapper) | Criminological network and link analysis, repeat-offender tracking |

---

## 2. Shared foundation: the feature store

Before describing each model, it's worth understanding what feeds them, since all four models draw from the same underlying data lake but transform it differently.

**Core entities**: FIR records, offenders, victims, locations (lat/lon + jurisdiction), incident timestamps, crime category/sub-category, and MO (modus operandi) tags.

**Engineered features** typically include:

- Temporal: hour of day, day of week, month, festival/holiday flags, season
- Spatial: district, taluk, police station jurisdiction, lat/lon, distance to nearest landmark
- Socio-economic: population density, literacy rate, unemployment rate, urbanization index (joined at district/taluk level from census data)
- Historical: rolling counts of crime by category over the past 7/30/90 days for the same area

- Text-derived: entities extracted from FIR narratives via NLP (weapon mentions, group size, relationship between victim and accused)

Each model below consumes a different subset and shape of this feature store.

---

## 3. Model 1 — XGBoost Risk Predictor

**Purpose**: Produce a district (or police-station-level) risk classification — High / Medium / Low — for upcoming time windows, so SCRB can pre-allocate resources before incidents spike rather than after.

**Why XGBoost**: It's a gradient-boosted tree ensemble, well suited to tabular data with mixed feature types (categorical district codes, continuous socio-economic indicators, cyclical time features), handles missing values gracefully, and produces feature importances that are useful for explaining *why* a district is flagged — important for investigator trust and for any future audit of the model's decisions.

**Input features**:

- District/station identifier (encoded)
- Time features (week of year, month, day-of-week, upcoming festival/event flags)
- Recent crime counts by category (lagged features, e.g. last 4 weeks)
- Socio-economic indicators (population density, unemployment, literacy)

**Training target**: Historical crime counts (or a binned risk label derived from them) for the *next* time window — e.g. "did this district see an above-threshold spike in the following week?"

**Output format** (as you specified):

```json
{
  "Bengaluru Urban": "HIGH",
  "Mysuru": "MEDIUM",
  "Hubli": "LOW"
}
```

In practice, it's worth also storing the underlying probability/score (e.g. 0–1) alongside the categorical label, so the dashboard can show a gradient rather than just three buckets, and so the High/Medium/Low thresholds can be tuned later without retraining.

**How it's used downstream**: The serving layer queries this model on a schedule (e.g. nightly or weekly) for every district, and the result colors the choropleth map (red/yellow/green) on the SCRB dashboard.

**Evaluation considerations**:

- Standard classification metrics (precision/recall/F1 per class) on a held-out time period — never randomly split, since this is a time-series problem and random splits leak future information into training.
- Calibration check: do districts labeled "HIGH" actually see more incidents in the following period than "LOW" districts, consistently across multiple time windows?
- Feature importance review with domain experts (SCRB analysts) to sanity-check that the model isn't keying off spurious correlations.

---

## 4. Model 2 — ST-DBSCAN Hotspot Detector

**Purpose:** Identify geographic clusters of crimes that are also concentrated in time — a "hotspot" isn't just a place with many crimes, it's a place *and* time window where crimes cluster together (e.g. chain-snatching near a market between 6–9 PM).

**Why ST-DBSCAN:** Standard DBSCAN clusters points by spatial density alone. ST-DBSCAN (Spatio-Temporal DBSCAN) extends this by requiring points to be close in *both* space and time to be considered part of the same cluster. This avoids merging two unrelated clusters that happen to occur at the same location but months apart, and avoids treating a citywide crime wave as one giant "hotspot."

**Input features:**

- Latitude and longitude of each incident
- Timestamp of each incident
- (Optionally) crime category, if you want category-specific hotspots rather than all-crime hotspots

**Key parameters to tune:**

- Spatial epsilon (e.g. 500m–2km depending on urban density)
- Temporal epsilon (e.g. 2–6 hour window, or same time-of-day across days)
- Minimum points to form a cluster (controls sensitivity vs. noise)

**Output format** (as you specified):

```json
[
  { "lat": 12.97, "lon": 77.59, "radius": 2, "intensity": "HIGH" },
  { "lat": 12.29, "lon": 76.63, "radius": 1, "intensity": "MEDIUM" }
]
```

The "intensity" field is typically derived from cluster density (points per unit area per unit time) — denser, more recent clusters get HIGH intensity.

**How it's used downstream**: Each cluster becomes a circle on the map, sized by radius and colored/animated by intensity — this is the "pulsing red-zone" visual described in the platform requirements. Re-running this model regularly (e.g. daily) lets the dashboard show how hotspots shift over the week, which is the core of the "time of day + location" layering requirement.

**Evaluation considerations**:

- Sensitivity analysis on epsilon parameters — too small and every cluster is noise, too large and the whole city becomes one cluster.
- Compare against known patrol logs: do officers already informally know about these hotspots? This is a useful sanity check before trusting the output.
- Track cluster stability over time — a hotspot that appears for one day and vanishes is less actionable than one that persists.

---

## 5. Model 3 — Isolation Forest Anomaly Detector

**Purpose**: Flag individual FIRs that look statistically unusual compared to the bulk of records — not because they're necessarily "important," but because they deviate from the patterns the model has learned, which makes them worth a second look by an investigator.

**Why Isolation Forest**: It's an unsupervised algorithm — useful here because there's no labeled "this FIR was anomalous" dataset to train against. It works by randomly partitioning the feature space and measuring how quickly a point gets isolated; outliers get isolated faster (fewer splits needed) and thus receive higher anomaly scores. It scales well to large numbers of records and doesn't assume any particular distribution of the data.

**Input features** (the "feature vector" per FIR):

- Crime category and sub-category (encoded)
- Time and location features
- Number of accused, number of victims
- Presence/absence of weapon, vehicle, specific MO tags (often derived via NLP from the FIR narrative)
- Case resolution time (if available historically)
- Any numeric fields like value of property stolen

**Output format** (as you specified):

```
json
```

```
{
  "FIR_4521": { "anomaly_score": 0.94 },
  "FIR_2210": { "anomaly_score": 0.12 }
}
```

**How it's used downstream:** FIRs above a chosen anomaly-score threshold get a visual flag on the investigator's dashboard (e.g. a badge or highlighted row). Importantly, this model does **not** label a case as "suspicious" or "criminal" — it only says "this combination of features is statistically rare in our historical data." The interpretation and follow-up is left entirely to the human investigator.

**Evaluation considerations:**

- Because there's no ground truth, evaluation relies on qualitative review: sample a set of high-scoring FIRs and have analysts assess whether the flags are useful (true positives in the sense of "worth a second look") versus noise.
- Watch for the model simply flagging *rare crime categories* as anomalous (e.g. a rarely-reported crime type will always score high) — this may or may not be the intended behavior, and should be discussed with domain experts.
- Re-train periodically, since "normal" patterns shift over time (seasonal crimes, new MOs).

---

## 6. Model 4 — Graph Algorithm Network Mapper

**Purpose:** Reveal relationships between offenders, victims, and locations that span multiple FIRs and jurisdictions — the core of the "criminological network and link analysis" capability. This is what turns isolated case records into a visible network of repeat offenders and shared associates.

**Why a graph approach:** Relational/tabular databases make it hard to ask questions like "who else has this person been connected to, across how many degrees of separation, and across how many districts?" A graph database (e.g. Neo4j) and graph algorithms are built precisely for this kind of traversal and pattern-matching.

**Input data:** Every FIR contributes nodes (offenders, victims, locations, vehicles if relevant) and edges (an offender *involved in* an incident, an incident *occurred at* a location, two offenders *named together* in the same FIR).

**Algorithms typically layered on top of the raw graph:**

- **Community detection** (e.g. Louvain or label propagation) — groups of individuals who repeatedly appear together, suggesting an organized group rather than coincidence.
- **Centrality measures** (degree, betweenness, PageRank) — identifies individuals who sit at the center of many connections, often the most useful starting points for an

investigation.

- **Link prediction** — suggests likely-but-unconfirmed connections based on shared attributes (same MO, same location, overlapping timeframes), which investigators can then verify.

**Output format** (as you specified):

```json
{
  "nodes": ["offender_1", "victim_3", "loc_7"],
  "edges": [
    { "source": "offender_1", "target": "victim_3" },
    { "source": "victim_3", "target": "loc_7" },
    { "source": "offender_1", "target": "loc_7" }
  ]
}
```

**How it's used downstream:** D3.js (or a similar force-directed graph library) renders this as an interactive network — clicking a node could expand it to show further connections, and edge thickness can represent the strength or frequency of a connection (e.g. number of shared incidents).

**Evaluation considerations:**

- This is exploratory/investigative tooling rather than a predictive model, so "accuracy" is less meaningful than *usefulness*. Evaluation should involve investigators using the tool on real cold cases or known organized-crime networks to see whether it surfaces connections they didn't already know about.
- Be careful with link-prediction suggestions — these should be clearly marked as "possible" connections requiring verification, not presented as established facts.

---

## 7. How the four models fit together in the serving layer

All four models run on different schedules and produce different shapes of output, so the serving/API layer's job is to normalize these into a consistent format the frontend can consume:

- **XGBoost** runs on a slower cadence (e.g. weekly) since district-level risk doesn't change hour to hour. Output: one JSON object per district.
- **ST-DBSCAN** runs more frequently (e.g. daily) to capture shifting hotspots. Output: a list of cluster objects.
- **Isolation Forest** runs incrementally as new FIRs are filed, scoring each new record. Output: a score per FIR ID, appended to a flagged-cases table.

- **Graph algorithm** updates incrementally as new FIRs add nodes/edges, but heavier algorithms (community detection) may run on a batch schedule (e.g. nightly) since they're computationally expensive over the whole graph.

A simple architectural pattern is to have each model as its own microservice (or scheduled job) that writes its output to a dedicated table/cache, and a single API gateway that the dashboard queries — the dashboard doesn't need to know which model produced what, only which endpoint to call for "district risk," "hotspots," "flagged FIRs," or "network graph for offender X."

---

## 8. Cross-cutting considerations

**Data quality is the binding constraint.** All four models depend on consistent, well-structured input data. If FIR narratives are inconsistent in how they record location, time, or MO, both the Isolation Forest and Graph Mapper will produce noisy results. Investing in NLP-based structuring of free-text FIRs (extracting standardized fields) will likely improve all four models more than tuning any individual model's hyperparameters.

**Predictive risk scoring and known limitations of this approach.** Models like the XGBoost risk predictor and ST-DBSCAN hotspot detector are forms of "predictive policing," an approach that has been studied extensively and criticized in research literature — primarily because historical crime data reflects historical *enforcement* patterns (where police already patrolled and recorded crime) as much as it reflects where crime actually occurs. This can create feedback loops where over-policed areas generate more recorded data, which the model then flags as high-risk, justifying more policing there. For a platform like this, it's worth building in:

- Regular audits comparing model outputs against independent indicators (e.g. victimization surveys where available)
- Treating model output as one input to resource allocation decisions, not an automatic trigger
- Avoiding individual-level predictive scoring (predicting that a *specific person* will commit a crime) — the models here are appropriately scoped to areas (districts, hotspot zones) and case records (FIRs), not to predicting individual future behavior

**Privacy and access control.** The graph/network model in particular handles sensitive personal data linking individuals across cases. Role-based access control should ensure this view is restricted to authorized investigators, with audit logging of who accessed which network views.

**Human-in-the-loop design.** Each model is framed here as producing a *signal for investigators*, not a *decision*. The dashboard design should reinforce this — e.g. anomaly flags and link-prediction suggestions should be visually distinct from confirmed facts, and the system should make it easy for an investigator to mark a flag as reviewed and dismiss false positives, which can also feed back into model refinement.

## 9. Suggested technology stack

| Layer | Suggested tools |
|---|---|
| Feature store / data lake | PostgreSQL + PostGIS (geospatial), or a cloud data warehouse |
| Graph storage | Neo4j (or similar property graph database) |
| Model training | Python: `xgboost`, `scikit-learn` (Isolation Forest), a spatiotemporal clustering library or custom ST-DBSCAN implementation |
| Graph algorithms | Neo4j Graph Data Science library, or `networkx`/`igraph` for offline analysis |
| Model serving | FastAPI (Python) microservices, or a single API gateway with scheduled batch jobs writing to cache tables |
| Frontend visualization | React + Leaflet/Mapbox (maps), D3.js (network graphs), a charting library for trend dashboards |

## 10. Summary

The four models cover four distinct analytical lenses — *where will risk concentrate* (XGBoost), *where and when are crimes clustering right now* (ST-DBSCAN), *which individual records look unusual* (Isolation Forest), and *who is connected to whom* (graph algorithm). Together, they transform the platform from a static reporting tool into the "Strategic Intelligence Hub" described in the original requirements — but their value depends heavily on data quality, careful threshold-setting, and keeping investigators firmly in control of how outputs translate into action.